

My Project

Generated by Doxygen 1.13.2

1 Projekto paleidimo instrukcija	1
1.1 Testai	1
1.1.1 Strategija 1, konteineris: vector, duomenu tipas: klase +	1
1.1.2 Strategija 1, konteineris: vector, duomenu tipas: struktura +	1
1.1.3 Optimizacija: O1	1
1.1.3.1 Strategija 1, konteineris: vector, duomenu tipas: struktura +	1
1.1.3.2 Strategija 1, konteineris: vector, duomenu tipas: klase +	2
1.1.4 Optimizacija: O2	2
1.1.4.1 Strategija 1, konteineris: vector, duomenu tipas: struktura +	2
1.1.4.2 Strategija 1, konteineris: vector, duomenu tipas: klase +	2
1.1.5 Optimizacija: O3	2
1.1.5.1 Strategija 1, konteineris: vector, duomenu tipas: struktura +	2
1.1.5.2 Strategija 1, konteineris: vector, duomenu tipas: klase +	2
1.1.6 Failu dydžiai:	3
1.1.6.1 su struktūromis	3
1.1.6.2 su klasėmis	3
1.2 "Rule of Five"	3
1.2.1 Kiekvieno metodo paaiškinimas	3
1.2.1.1 1. Destruktorius	3
1.2.1.2 2. Kopijavimo konstruktorius	4
1.2.1.3 3. Kopijavimo priskyrimo operatorius	4
1.2.1.4 4. Perkėlimo konstruktorius	4
1.2.1.5 5. Perkėlimo priskyrimo operatorius	5
1.3 Operatorių perdengimas (<< ir >>)	5
1.3.1 Kas tai yra?	5
1.3.2 Perdengimų paaiškinimai	5
1.3.2.1 1. Įvesties operatorius >>	5
1.3.2.2 2. Išvesties operatorius <<	5
1.4 Santrauka	6
1.5 Abstrakčios klasės	6
1.5.1 Kas yra abstrakti klasė?	6
1.5.2 Dabartinė Zmogus klasės implementacija	6
1.5.2.1 Komentaras apie implementaciją:	6
2 Hierarchical Index	7
2.1 Class Hierarchy	7
3 Class Index	9
3.1 Class List	9
4 File Index	11
4.1 File List	11
5 Class Documentation	13

5.1 Studentas Class Reference	13
5.1.1 Member Function Documentation	13
5.1.1.1 prisistatymas()	13
5.2 Zmogus Class Reference	14
6 File Documentation	15
6.1 funkcijos.h	15
6.2 pagalbines.h	19
Index	21

Chapter 1

Projekto paleidimo instrukcija

1. Naudodami git, galite klonuoti si projekta su komanda `git clone https://github.com/Liveta/Liveta/Objektinis-Programavimas.git`
2. Sukurkite projekto .exe faila, paleisdami `make` komanda projekto direktorijoje
3. Paleiskite projekta, paleisdami `./vektoriai` komanda

1.1 Testai

1.1.1 Strategija 1, konteineris: vector, duomenu tipas: klase +

Studentu kiekis	failo skaitymas	failo rikiavimas	failo filtravimas
1000	0.0078709 s	0.0026605 s	0.0013088 s
10000	0.0733157 s	0.0227285 s	0.0064966 s
100000	0.372429 s	0.237906 s	0.0603885 s
1000000	3.59903 s	3.10719 s	0.637031 s
10000000	43.0755 s	47.4339 s	7.08653 s

1.1.2 Strategija 1, konteineris: vector, duomenu tipas: struktura +

Studentu kiekis	failo skaitymas	failo rikiavimas	failo filtravimas
1000	0.0056594 s	0.0023562 s	0.0012052 s
10000	0.0455578 s	0.0188143 s	0.0072981 s
100000	0.443563 s	0.249256 s	0.0669365 s
1000000	3.57023 s	3.25399 s	0.618061 s
10000000	40.5932 s	48.5612 s	7.47094 s

1.1.3 Optimizacija: O1

1.1.3.1 Strategija 1, konteineris: vector, duomenu tipas: struktura +

Studentu kiekis	failo skaitymas	failo rikiavimas	failo filtravimas
1000	0.0049481 s	0.0009946 s	0.0006666 s
10000	0.0411275 s	0.0077642 s	0.0031267 s
100000	0.277411 s	0.0936089 s	0.0415893 s
1000000	2.83898 s	1.03747 s	0.353189 s
10000000	28.0223 s	19.4457 s	4.5401 s

1.1.3.2 Strategija 1, konteineris: vector, duomenų tipas: klase +

Studentu kiekis	failo skaitymas	failo rikiavimas	failo filtravimas
1000	0.0051615 s	0.0010357 s	0.0007957 s
10000	0.0389726 s	0.0073931 s	0.0028394 s
100000	0.262372 s	0.0859877 s	0.0310893 s
1000000	2.54161 s	1.04326 s	0.325477 s
10000000	32.3925 s	23.4776 s	4.18133 s

1.1.4 Optimizacija: O2

1.1.4.1 Strategija 1, konteineris: vector, duomenų tipas: struktura +

Studentu kiekis	failo skaitymas	failo rikiavimas	failo filtravimas
1000	0.0051699 s	0.0006635 s	0.000436 s
10000	0.0379266 s	0.0094879 s	0.0039316 s
100000	0.263982 s	0.0933424 s	0.0344979 s
1000000	2.63518 s	1.00792 s	0.330343 s
10000000	29.246 s	21.3922 s	4.35245 s

1.1.4.2 Strategija 1, konteineris: vector, duomenų tipas: klase +

Studentu kiekis	failo skaitymas	failo rikiavimas	failo filtravimas
1000	0.0048522 s	0.0007731 s	0.0005992 s
10000	0.0340502 s	0.0065104 s	0.0031781 s
100000	0.253193 s	0.0793256 s	0.0365522 s
1000000	2.57298 s	1.02315 s	0.346465 s
10000000	28.1724 s	18.8565 s	4.2299 s

1.1.5 Optimizacija: O3

1.1.5.1 Strategija 1, konteineris: vector, duomenų tipas: struktura +

Studentu kiekis	failo skaitymas	failo rikiavimas	failo filtravimas
1000	0.0060205 s	0.0008658 s	0.0004774 s
10000	0.0377913 s	0.0079911 s	0.0042306 s
100000	0.265273 s	0.0817811 s	0.0328203 s
1000000	3.25478 s	1.06061 s	0.341586 s
10000000	30.0521 s	19.1179 s	4.42345 s

1.1.5.2 Strategija 1, konteineris: vector, duomenų tipas: klase +

Studentu kiekis	failo skaitymas	failo rikiavimas	failo filtravimas
1000	0.0050069 s	0.0008638 s	0.0007102 s
10000	0.0350589 s	0.007263 s	0.0041975 s
100000	0.267193 s	0.0846852 s	0.0300512 s
1000000	2.60526 s	1.03718 s	0.343012 s
10000000	29.0081 s	18.8643 s	4.48713 s

1.1.6 Failu dydžiai:

1.1.6.1 su struktūromis

vektoriai.exe : 622KB vektoriai_O1.exe : 319KB vektoriai_O2.exe : 298KB vektoriai_O3.exe : 294KB

1.1.6.2 su klasėmis

vektoriai.exe : 712KB vektoriai_O1.exe : 349KB vektoriai_O2.exe : 317KB vektoriai_O3.exe : 310KB

1.2 "Rule of Five"

Kas yra "Rule of Five"?

C++ kalboje, jei klasė valdo išteklius (pvz., dinaminę atmintį, failus ar pan.), reikia apibrėžti penkis specialius metodus:

1. Destruktorius
2. Kopijavimo konstruktorius
3. Kopijavimo priskyrimo operatorius
4. Perkėlimo konstruktorius
5. Perkėlimo priskyrimo operatorius

1.2.1 Kiekvieno metodo paaiškinimas

1.2.1.1 1. Destruktorius

Paskirtis:

- Atlaisvina išteklius, kai objektas sunaikinamas.

Pavyzdys:

```
Studentas::~Studentas() {
    vardas.clear();
    pavarde.clear();
    pazymiai.clear();
}
```

1.2.1.2 2. Kopijavimo konstruktorius

Paskirtis:

- Sukuria naują objektą kaip tikslią kito objekto kopiją.

Panaudojimas:

- Naudojamas, kai reikia nukopijuoti objektą, kad būtų sukurtas nepriklausomas jo dublikatas.

Pavyzdys:

```
Studentas::Studentas(const Studentas& originalas) {  
    pavarde = originalas.pavarde;  
    vardas = originalas.vardas;  
    pazymiai = originalas.pazymiai;  
    egzamino_pazymys = originalas egzamino_pazymys;  
    vidurkis = originalas.vidurkis;  
    mediana = originalas.mediana;  
}
```

1.2.1.3 3. Kopijavimo priskyrimo operatorius

Paskirtis:

- Pakeičia esamo objekto turinį kitu objektu.

Panaudojimas:

- $a = b$; kai abu objektai jau egzistuoja.

Pavyzdys:

```
Studentas& Studentas::operator=(const Studentas& originalas) {  
    if (this != &originalas) {  
        pavarde = originalas.pavarde;  
        vardas = originalas.vardas;  
        pazymiai = originalas.pazymiai;  
        egzamino_pazymys = originalas egzamino_pazymys;  
        vidurkis = originalas.vidurkis;  
        mediana = originalas.mediana;  
    }  
    return *this;  
}
```

1.2.1.4 4. Perkėlimo konstruktorius

Paskirtis:

- Sukuria naują objektą, "pavogdamas" išteklius iš kito objekto.

Panaudojimas:

- Efektyvus darbas su laikiniais objektais.

Pavyzdys:

```
Studentas::Studentas(Studentas&& originalas) noexcept {  
    pavarde = move(originalas.pavarde);  
    vardas = move(originalas.vardas);  
    pazymiai = move(originalas.pazymiai);  
    egzamino_pazymys = originalas egzamino_pazymys;  
    vidurkis = originalas.vidurkis;  
    mediana = originalas.mediana;  
  
    originalas egzamino_pazymys = 0;  
    originalas.vidurkis = 0;  
    originalas.mediana = 0;  
}
```


1.2.1.5 5. Perkėlimo priskyrimo operatorius

Paskirtis:

- Pakeičia objekto turinį, "pavogdamas" iš kito objekto išteklius.

Panaudojimas:

- `a = move(b);`

Pavyzdys:

```
Studentas& Studentas::operator=(Studentas&& originalas) noexcept {
    if (this != &originalas) {
        pavarde = move(originalas.pavarde);
        vardas = move(originalas.vardas);
        pazymiai = move(originalas.pazymiai);
        egzamino_pazymys = originalas.egzamino_pazymys;
        vidurkis = originalas.vidurkis;
        mediana = originalas.mediana;

        originalas.egzamino_pazymys = 0;
        originalas.vidurkis = 0;
        originalas.mediana = 0;
    }
    return *this;
}
```

1.3 Operatorių perdengimas (<< ir >>)

1.3.1 Kas tai yra?

Operatorių perdengimas leidžia nustatyti, kaip objektai yra skaitomi iš srauto (>>) ir spausdinami į srautą (<<). Tai leidžia naudoti paprastą sintaksę kaip `cout << objektas; arba cin >> objektas;`.

1.3.2 Perdengimų paaiškinimai

1.3.2.1 1. Įvesties operatorius >>

Paskirtis:

- Nuskaito `Studentas` objekto duomenis iš vartotojo arba failo.

Pavyzdys:

```
istream& operator>>(istream& isvesties_vieta, Studentas& studentas) {
    cout << "Įveskite vardą: ";
    isvesties_vieta >> studentas.vardas;

    cout << "Įveskite pavardę: ";
    isvesties_vieta >> studentas.pavarde;

    cout << "Įveskite pažymių kiekį: ";
    int kiekis;
    isvesties_vieta >> kiekis;

    studentas.pazymiai.clear();
    cout << "Įveskite pažymius: ";
    for (int i = 0; i < kiekis; ++i) {
        int pazymys;
        isvesties_vieta >> pazymys;
        studentas.pazymiai.push_back(pazymys);
    }

    cout << "Įveskite egzamino pažymį: ";
    isvesties_vieta >> studentas.egzamino_pazymys;

    studentas.vidurkio_skaiciavimas();
    studentas.medianos_skaiciavimas();

    return isvesties_vieta;
}
```

1.3.2.2 2. Išvesties operatorius <<

Paskirtis:

- Išveda `Studentas` objekto duomenis į ekraną arba failą.

Pavyzdys:

```
ostream& operator<<(ostream& isvesties_vieta, const Studentas& studentas) {
    isvesties_vieta << studentas vardas << " " << studentas.pavarde << " | Pažymiai: ";
    for (int pazymys : studentas.pazymiai) {
        isvesties_vieta << pazymys << " ";
    }
    isvesties_vieta << " | Egzaminas: " << studentas.egzamino_pazymys;
    isvesties_vieta << " | Vidurkis: " << studentas.vidurkis;
    isvesties_vieta << " | Mediana: " << studentas.mediana;
    return isvesties_vieta;
}
```

1.4 Santrauka

Koncepcija	Paskirtis	Pavyzdys
Destruktorius	Atlaisvina išteklius sunaikinant objektą	<code>~Studentas()</code>
Kopijavimo konstruktorius	Sukuria naują kopiją	<code>Studentas b = a;</code>
Kopijavimo priskyrimas	Pakeičia objekto turinį kitu	<code>b = a;</code>
Perkėlimo konstruktorius	Pavogia išteklius iš kito objekto	<code>Studentas b = move(a);</code>
Perkėlimo priskyrimas	Pavogia išteklius per priskyrimą	<code>b = move(a);</code>
Išvesties operatorius <code><<</code>	Išveda objektą į ekraną arba failą	<code>cout << studentas;</code>
Įvesties operatorius <code>>></code>	Nuskaityti objektą iš vartotojo ar failo	<code>cin >> studentas;</code>

1.5 Abstrakčios klasės

1.5.1 Kas yra abstrakti klasė?

Abstrakti klasė yra tokia klasė, kuri aprašo bendras sąvokas ir turi bent vieną abstraktų metodą (be implementacijos). Tokia klasė negali būti sukuriamą tiesiogiai, ji naudojama kaip bazė kitoms klasėms paveldėti ir konkretizuoti elgesį. Abstrakčios klasės tikslas - apibrėžti, kokias funkcijas turi įgyvendinti paveldėtos klasės, paliekant realizacijos detales joms pačioms.

1.5.2 Dabartinė Zmogus klasės implementacija

```
class Zmogus{
public:
    virtual void prisistatymas() = 0;
    virtual ~Zmogus() = default;
};
```

1.5.2.1 Komentaras apie implementaciją:

- `prisistatymas()` yra **grynas virtualus metodas** (`= 0`), kuris reiškia, kad kiekviena paveldinti klasė privalo įgyvendinti savo prisistatymo logiką.
- `virtual ~Zmogus() = default;` garantuoja, kad sunaikinant išvestinės klasės objektą bus teisingai kviečiamas destruktoriaus.

Ši bazinė struktūra yra kaip pagrindas, leidžiantis kurti įvairias `Zmogus` klases, tokias kaip `Studentas`, `Darbuotojas` ir pan., priversdama jas implementuoti savo `prisistatymas()` metodo.

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	14
Studentas	13

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	13
Zmogus	14

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

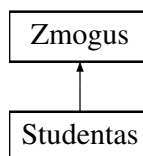
funkcijos.h	15
pagalbines.h	19

Chapter 5

Class Documentation

5.1 Studentas Class Reference

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** (string vardass, string pavardee, vector< int > pazymiai, int egzamino_pazymyss)
- **Studentas** (const [Studentas](#) &originalas)
- [Studentas](#) & **operator=** (const [Studentas](#) &originalas)
- **Studentas** ([Studentas](#) &&originalas) noexcept
- [Studentas](#) & **operator=** ([Studentas](#) &&originalas) noexcept
- void [prisistatymas](#) () override
- void **vidurkio_skaiciavimas** ()
- void **medianos_skaiciavimas** ()

Public Attributes

- string **pavarde**
- string **vardas**
- vector< int > **pazymiai**
- int **egzamino_pazymys**
- double **vidurkis**
- double **mediana**

Friends

- ostream & **operator<<** (ostream &isvesties_vieta, const [Studentas](#) &studentas)
- istream & **operator>>** (istream &isvesties_vieta, [Studentas](#) &studentas)

5.1.1 Member Function Documentation

5.1.1.1 prisistatymas()

```
void Studentas::prisistatymas () [override], [virtual]
```

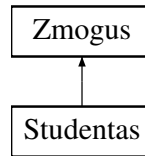
Implements [Zmogus](#).

The documentation for this class was generated from the following files:

- funkcijos.h
- funkcijos.cpp

5.2 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- virtual void **prisistatymas** ()=0

The documentation for this class was generated from the following file:

- funkcijos.h

Chapter 6

File Documentation

6.1 funkcijos.h

```
00001 #include "pagalbines.h"
00002 #ifndef FUNKCIJOS_H
00003 #define FUNKCIJOS_H
00004
00005 class Zmogus{
00006     public:
00007         virtual void prisistatymas() = 0;
00008         virtual ~Zmogus() = default;
00009 };
00010
00011 class Studentas : public Zmogus{
00012     public:
00013         string pavarde;
00014         string vardas;
00015         vector<int> pazymiai;
00016         int egzamino_pazymys;
00017         double vidurkis;
00018         double mediana;
00019
00020         Studentas(string vardass, string pavardee, vector<int> pazymiaai, int egzamino_pazymyss);
00021         Studentas(const Studentas& originalas);
00022         Studentas& operator=(const Studentas& originalas);
00023         Studentas(Studentas&& originalas) noexcept;
00024         Studentas& operator=(Studentas&& originalas) noexcept;
00025         Studentas();
00026         ~Studentas();
00027         friend ostream& operator<<(ostream& isvesties_vieta, const Studentas& studentas);
00028         friend istream& operator>>(istream& isvesties_vieta, Studentas& studentas);
00029         void prisistatymas() override;
00030         void vidurkio_skaiciavimas();
00031         void medianos_skaiciavimas();
00032 };
00033
00034 extern int studentu_kiekis;
00035 extern vector<string> vardai;
00036 extern vector<string> pavardes;
00037 extern vector<Studentas> studentai;
00038 extern vector<double> operaciju_laikai;
00039 extern int operaciju_kiekis;
00040
00041 void failo_su_studentais_generavimas();
00042 int pasirinkimo_pavertimas_i_reiksme(int failo_generavimo_pasirinkimas);
00043 void failo_generavimas(int studentu_kiekis, int pazymiu_kiekis);
00044 int vieno_pazymio_sugeneravimas();
00045
00046 void studento_duomenu_gavimas(vector<Studentas> &studentai);
00047 int studento_duomenu_surasymo_pasirinkimas();
00048 double laiku_vidurkio_skaiciavimas(vector<double> &operaciju_laikai);
00049
00050 void studento_duomenu_printinimas(vector<Studentas> &studentai);
00051 void studento_duomenu_rikiavimas(vector<Studentas> &studentai);
00052 bool rikiuoti_pagal_varda(Studentas &a, Studentas &b);
00053 bool rikiuoti_pagal_pavarde(Studentas &a, Studentas &b);
00054 bool rikiuoti_pagal_vidurkis(Studentas &a, Studentas &b);
00055 bool rikiuoti_pagal_mediana(Studentas &a, Studentas &b);
00056 bool baigiasi_su_txt(string failo_pavadinimas);
00057
00058 void studento_vardo_ir_pavardes_gavimas(Studentas &laikinas_studentas);
00059 void studento_pazymiu_gavimas(Studentas &laikinas_studentas);
00060 void studento_egzamino_pazymio_gavimas(Studentas &laikinas_studentas);
00061
```

```

00062 void tikrinimas_ar_pavyko_atidaryti_faila(string& failo_pavadinimas);
00063
00064 void studento_pazymiu_ir_egzaminu_generavimas(Studentas &laikinas_studentas);
00065 int gauk_kiek_pazymiu_sugeneruoti();
00066 void studento_vardo_ir_pavardes_generavimas(Studentas &laikinas_studentas);
00067
00068 void visi_duomenys_surasomi_ranka();
00069 void surasoma_ranka_isskyrus_pazymius();
00070 void viskas_generuojama_atstitiktinai();
00071
00072
00073 template <typename konteineris>
00074 void studentu_medianos_skaiciavimas(konteineris &studentai){
00075     for (auto it = studentai.begin(); it != studentai.end(); ++it){
00076         it->medianos_skaiciavimas();
00077     }
00078 }
00079 template <typename konteineris>
00080 void duomenu_surasymas_i_faila(konteineris studentai, string failo_pavadinimas){
00081     ofstream failas(failo_pavadinimas);
00082     failas << setw(18) << left << "Vardas" << setw(18) << left << "Pavarde" << setw(20) << left << "Galutinis
(Vid.)" << setw(20) << left << "Galutinis (Med.)" << endl;
00083     failas << "-----" << endl;
00084
00085     for (auto iteratorius = studentai.begin(); iteratorius != studentai.end(); ++iteratorius) {
00086         failas << std::setw(18) << std::left << iteratorius->vardas
00087             << std::setw(18) << std::left << iteratorius->pavarde
00088             << std::setw(20) << std::left << std::fixed << std::setprecision(2) <<
iteratorius->vidurkis
00089             << std::setw(20) << iteratorius->mediana
00090             << std::endl;
00091     }
00092     failas.close();
00093 }
00094
00095
00096 template <typename konteineris>
00097 void studento_vidurkio_skaiciavimas(konteineris &studentai){
00098
00099     for (auto iteratorius = studentai.begin(); iteratorius != studentai.end(); ++iteratorius) {
00100         iteratorius->vidurkio_skaiciavimas();
00101     }
00102 }
00103
00104 template <typename konteineris>
00105 void studento_duomenu_is_failo_susirasymas(konteineris&studentai, string& failo_pavadinimas){
00106     auto pradzia = std::chrono::high_resolution_clock::now();
00107
00108     ifstream failas(failo_pavadinimas);
00109     string antrastine_eilute;
00110     getline(failas, antrastine_eilute);
00111
00112     string eilute;
00113
00114     while(getline(failas, eilute)){
00115         Studentas laikinas_studentas;
00116         stringstream eil(eilute);
00117         eil >> laikinas_studentas.vardas >> laikinas_studentas.pavarde;
00118         int pazymys;
00119         while (eil >> pazymys){
00120             laikinas_studentas.pazymiai.push_back(pazymys);
00121         }
00122         laikinas_studentas.egzamino_pazymys = laikinas_studentas.pazymiai.back();
00123         laikinas_studentas.pazymiai.pop_back();
00124         studentai.push_back(laikinas_studentas);
00125     }
00126     operaciju_kiekis += 1;
00127
00128     auto pabaiga = std::chrono::high_resolution_clock::now();
00129     std::chrono::duration<double> trukme = pabaiga - pradzia;
00130     cout << endl << "duomenu_is_failo_nuskaitymas_uztruko: " << trukme.count() << " s" << endl;
00131     operaciju_laikai.push_back(trukme.count());
00132     failas.close();
00133 }
00134
00135 template <typename konteineris>
00136 void studento_duomenu_skaitymas_is_failo(konteineris& studentai){
00137     string failo_pavadinimas;
00138     cout << "Iveskite norimo nuskaityti failo pavadinima: " << endl;
00139     cin >> failo_pavadinimas;
00140
00141     while (true){
00142         try{
00143             tikrinimas_ar_pavyko_atidaryti_faila(failo_pavadinimas);
00144             break;
00145         }catch(exception &eroras){
00146             cout << eroras.what() << endl;

```

```

00147         cin » failo_pavadinimas;
00148     }
00149 }
00150 cout « "Failas atidarytas sėkmingai!" « endl;
00151 studento_duomenu_is_failo_susirasymas(studentai, failo_pavadinimas);
00152
00153 }
00154
00155 template <typename konteineris>
00156 void strategija_1(){
00157     konteineris visi;
00158     konteineris vargsiukai;
00160     konteineris kietiakiiai;
00161
00162     studento_duomenu_skaitymas_is_failo(visi);
00163     studento_vidurkio_skaiciavimas(visi);
00164
00165     auto rikiavimo_pradzia = std::chrono::high_resolution_clock::now();
00166
00167     if constexpr (is_same_v<konteineris, list<Studentas>)> {
00168         visi.sort(rikiuoti_pagal_vidurkis);
00169     } else{
00170         sort(visi.begin(), visi.end(), rikiuoti_pagal_vidurkis);
00171     }
00172
00173     auto rikiavimo_pabaiga = std::chrono::high_resolution_clock::now();
00174     std::chrono::duration<double> rikiavimo_trukme = rikiavimo_pabaiga - rikiavimo_pradzia;
00175     cout « endl « "Duomenu rikiavimas nuo didžiausio vidurkio iki mažiausio užtruko: " «
00176         rikiavimo_trukme.count() « " s" « endl;
00177
00178     auto isskirstymo_pradzia = std::chrono::high_resolution_clock::now();
00179
00180     for (auto it = visi.begin(); it != visi.end(); ++it) {
00181         if (it->vidurkis >= 5) {
00182             kietiakiiai.push_back(*it);
00183         } else {
00184             vargsiukai.push_back(*it);
00185         }
00186     }
00187
00188     auto isskirstymo_pabaiga = std::chrono::high_resolution_clock::now();
00189     std::chrono::duration<double> isskirstymo_trukme = isskirstymo_pabaiga - isskirstymo_pradzia;
00190     cout « endl « "Duomenu isskirstymas į kietiakus ir vargsiukus užtruko: " «
00191         isskirstymo_trukme.count() « " s" « endl;
00192
00193     string failo_pavadinimo_pabaiga = ".txt";
00194     string failo_pavadinimas_kieti = "kietiakiiai" + to_string(kietiakiiai.size()) +
00195         failo_pavadinimo_pabaiga;
00196     string failo_pavadinimas_vargsai = "vargsiukai" + to_string(vargsiukai.size()) +
00197         failo_pavadinimo_pabaiga;
00198
00199     studento_vidurkio_skaiciavimas(kietiakiiai);
00200     studento_vidurkio_skaiciavimas(vargsiukai);
00201     studentu_medianos_skaiciavimas(kietiakiiai);
00202     studentu_medianos_skaiciavimas(vargsiukai);
00203     duomenu_surasymas_i_faila(kietiakiiai, failo_pavadinimas_kieti);
00204     duomenu_surasymas_i_faila(vargsiukai, failo_pavadinimas_vargsai);
00205 }
00206
00207 template <typename konteineris>
00208 void isrinkVargsiukus(konteineris& visi, konteineris& vargsiukai){
00209     if constexpr (std::is_same_v<konteineris, std::list<Studentas>)> {
00210         // Su list trinama iš priekio į galą
00211         auto it = visi.begin();
00212         while (it != visi.end()) {
00213             if (it->vidurkis < 5) {
00214                 vargsiukai.push_back(*it);
00215                 it = visi.erase(it);
00216             } else {
00217                 ++it;
00218             }
00219         }
00220     } else {
00221         // Su vector / deque - trinama iš galo į priekį, kad mažiau perstumdinėtų
00222         for (auto it = visi.end(); it != visi.begin(); ) {
00223             --it;
00224             if (it->vidurkis < 5) {
00225                 vargsiukai.push_back(*it);
00226                 it = visi.erase(it); // erase grąžina iteratorių į sekantį po ištrinto
00227             }
00228         }
00229     }
00230 }
00231
00232 template <typename konteineris>
00233 void strategija_2(){

```

```

00230
00231     konteineris visi;
00232     konteineris vargsiukai;
00233
00234     studento_duomenu_skaitymas_is_failo(visi);
00235     studento_vidurkio_skaiciavimas(visi);
00236
00237     auto rikiavimo_pradzia = std::chrono::high_resolution_clock::now();
00238
00239     if constexpr (is_same_v<konteineris, list<Studentas>)> {
00240         visi.sort(rikiuoti_pagal_vidurkis);
00241     } else {
00242         sort(visi.begin(), visi.end(), rikiuoti_pagal_vidurkis);
00243     }
00244
00245     auto rikiavimo_pabaiga = std::chrono::high_resolution_clock::now();
00246     std::chrono::duration<double> rikiavimo_trukme = rikiavimo_pabaiga - rikiavimo_pradzia;
00247     cout << endl << "Duomenu rikiavimas nuo didziausio vidurkio iki maziausio uztruko: " <<
    rikiavimo_trukme.count() << " s" << endl;
00248
00249     auto isskirstymo_pradzia = std::chrono::high_resolution_clock::now();
00250
00251     isrinkVargsiukus(visi, vargsiukai);
00252
00253     auto isskirstymo_pabaiga = std::chrono::high_resolution_clock::now();
00254
00255     if constexpr (is_same_v<konteineris, list<Studentas>)> {
00256         vargsiukai.sort(rikiuoti_pagal_vidurkis);
00257     } else {
00258         sort(vargsiukai.begin(), vargsiukai.end(), rikiuoti_pagal_vidurkis);
00259     }
00260
00261     std::chrono::duration<double> isskirstymo_trukme = isskirstymo_pabaiga - isskirstymo_pradzia;
00262     cout << endl << "Duomenu isskirstymas i kietiakes ir vargsiukus uztruko: " <<
    isskirstymo_trukme.count() << " s" << endl;
00263
00264     string failo_pavadinimo_pabaiga = ".txt";
00265     string failo_pavadinimas_kieti = "kietiakes" + to_string(visi.size()) + failo_pavadinimo_pabaiga;
00266     string failo_pavadinimas_vargsai = "vargsiukai" + to_string(vargsiukai.size()) +
    failo_pavadinimo_pabaiga;
00267
00268     studento_vidurkio_skaiciavimas(visi);
00269     studento_vidurkio_skaiciavimas(vargsiukai);
00270     studentu_medianos_skaiciavimas(visi);
00271     studentu_medianos_skaiciavimas(vargsiukai);
00272     duomenu_surasymas_i_faila(visi, failo_pavadinimas_kieti);
00273     duomenu_surasymas_i_faila(vargsiukai, failo_pavadinimas_vargsai);
00274 }
00275 template <typename konteineris>
00276 void strategija_3(){
00277
00278     konteineris visi;
00279     studento_duomenu_skaitymas_is_failo(visi);
00280     studento_vidurkio_skaiciavimas(visi);
00281
00282     auto rikiavimo_pradzia = std::chrono::high_resolution_clock::now();
00283     if constexpr (is_same_v<konteineris, list<Studentas>)> {
00284         visi.sort(rikiuoti_pagal_vidurkis);
00285     } else {
00286         sort(visi.begin(), visi.end(), rikiuoti_pagal_vidurkis);
00287     }
00288     auto rikiavimo_pabaiga = std::chrono::high_resolution_clock::now();
00289     std::chrono::duration<double> rikiavimo_trukme = rikiavimo_pabaiga - rikiavimo_pradzia;
00290     cout << "\nDuomenu rikiavimas nuo didziausio vidurkio iki maziausio uztruko: " <<
    rikiavimo_trukme.count() << " s" << endl;
00291
00292     auto isskirstymo_pradzia = std::chrono::high_resolution_clock::now();
00293
00294     // Partitioning students into two groups using stable_partition
00295     auto it = stable_partition(visi.begin(), visi.end(), [](const Studentas& s) {
00296         return s.vidurkis >= 5;
00297     });
00298
00299     konteineris kietiakes(visi.begin(), it);
00300     konteineris vargsiukai(it, visi.end());
00301
00302     auto isskirstymo_pabaiga = std::chrono::high_resolution_clock::now();
00303     std::chrono::duration<double> isskirstymo_trukme = isskirstymo_pabaiga - isskirstymo_pradzia;
00304     cout << "\nDuomenu isskirstymas i kietiakes ir vargsiukus uztruko: " << isskirstymo_trukme.count() <<
    " s" << endl;
00305
00306     string failo_pavadinimo_pabaiga = ".txt";
00307     string failo_pavadinimas_kieti = "kietiakes" + to_string(kietiakes.size()) +
    failo_pavadinimo_pabaiga;
00308     string failo_pavadinimas_vargsai = "vargsiukai" + to_string(vargsiukai.size()) +
    failo_pavadinimo_pabaiga;
00309

```

```
00310     studento_vidurkio_skaiciavimas(kietiakiai);
00311     studento_vidurkio_skaiciavimas(vargsiukai);
00312     studentu_medianos_skaiciavimas(kietiakiai);
00313     studentu_medianos_skaiciavimas(vargsiukai);
00314     duomenu_surasymas_i_faila(kietiakiai, failo_pavadinimas_kieti);
00315     duomenu_surasymas_i_faila(vargsiukai, failo_pavadinimas_vargsai);
00316 }
00317
00318 #endif
```

6.2 pagalbines.h

```
00001 #include <iostream>
00002 #include <iomanip>
00003 #include <vector>
00004 #include <fstream>
00005 #include <string>
00006 #include <algorithm>
00007 #include <limits>
00008 #include <stdlib.h>
00009 #include <ctime>
00010 #include <fstream>
00011 #include <sstream>
00012 #include <functional>
00013 #include <cctype>
00014 #include <chrono>
00015 #include <list>
00016 #include <deque>
00017 #include <cmath>
00018 #include <iterator>
00019 #include <type_traits>
00020
00021 using std:: cout;
00022 using std:: cin;
00023 using std:: endl;
00024 using std:: string;
00025 using std:: vector;
00026 using std:: setw;
00027 using std:: left;
00028 using std:: right;
00029 using std:: setprecision;
00030 using std:: fixed;
00031 using std:: sort;
00032 using std:: numeric_limits;
00033 using std:: streamsize;
00034 using std:: srand;
00035 using std:: rand;
00036 using std:: time;
00037 using std:: ifstream;
00038 using std:: ofstream;
00039 using std:: getline;
00040 using std:: stringstream;
00041 using std:: greater;
00042 using std:: any_of;
00043 using :: isdigit;
00044 using std:: atoi;
00045 using :: tolower;
00046 using std:: transform;
00047 using std:: invalid_argument;
00048 using std:: runtime_error;
00049 using std:: exception;
00050 using std:: to_string;
00051 using std:: list;
00052 using std:: deque;
00053 using std:: is_same_v;
00054 using std:: move;
00055 using std:: ostream;
00056 using std:: istream;
00057 using std:: chrono::round;
```


Index

prisistatymas

Studentas, [13](#)

Projekto paleidimo instrukcija, [1](#)

Studentas, [13](#)

prisistatymas, [13](#)

Zmogus, [14](#)